

DineFinder

Restaurant Discovery Platform

Complete Project Documentation

Version 1.0 | June 2026

Table of Contents

Right-click the TOC and select "Update Field" to refresh page numbers

Table of Contents	2
1. Project Planning & Management	5
1.1 Project Proposal.....	5
Project Objectives	5
Project Scope	5
1.2 Project Plan & Timeline	5
Development Phases	6
Key Milestones.....	6
1.3 Task Assignment & Roles	6
1.4 Risk Assessment & Mitigation	6
1.5 Key Performance Indicators.....	7
2. Literature Review	7
Feedback & Evaluation.....	7
Suggested Improvements	7
Grading Criteria	8
3. Requirements Gathering.....	8
3.1 Stakeholder Analysis.....	8
3.2 User Stories & Use Cases.....	8
Primary Use Cases.....	8
User Stories	9
3.3 Functional Requirements	9
3.4 Non-Functional Requirements	10
4. System Analysis & Design	10
4.1 Problem Statement & Objectives	10
Design Objectives.....	11
4.2 Software Architecture.....	11
Architecture Components.....	11
Technology Stack.....	11

4.3 Database Design & Data Modeling.....	12
Entity-Relationship Model	12
Logical Schema.....	12
4.4 Data Flow Diagrams	12
Context-Level DFD (Level 0)	12
Level 1 DFD.....	13
Data Stores	13
4.5 Sequence Diagrams	13
Restaurant Search Sequence	13
User Authentication Sequence	13
Smart Assistant Sequence.....	14
4.6 Activity Diagrams.....	14
Restaurant Discovery Process	14
Save Restaurant Process	14
4.7 State Diagrams.....	14
User Authentication State.....	14
Restaurant Card State	14
4.8 Class Diagram	15
4.9 UI/UX Design & Prototyping	15
Design Principles	15
Color Scheme	16
Typography.....	16
4.10 Deployment Diagram.....	16
4.11 Component Diagram	16
4.12 API Documentation.....	17
Internal API Endpoints (Express Proxy)	17
External API Integration (Yelp Fusion).....	17
Data Transformation	17
5. Implementation	17
5.1 Source Code Structure.....	17
Directory Organization	18
Coding Standards	18
5.2 Version Control	18
Branching Strategy	18

5.3 Deployment & Execution.....	19
System Requirements	19
Installation Steps.....	19
Production Build	19
6. Testing & Quality Assurance.....	19
6.1 Test Cases & Test Plan.....	19
Authentication Test Cases	20
Restaurant Discovery Test Cases	20
Smart Assistant Test Cases	20
Admin Dashboard Test Cases	20
6.2 Bug Reports	21
Resolved Issues	21
7. Final Presentation & Reports	21
7.1 User Manual	21
Getting Started.....	21
For Guest Users.....	21
For Registered Users	22
For Administrators	22
7.2 Technical Documentation	22
API Configuration	22
Environment Variables	22
Performance Considerations.....	23
Security Notes	23

1. Project Planning & Management

1.1 Project Proposal

DineFinder is a full-stack web application designed to revolutionize the way users discover, explore, and manage restaurant choices. In an era where dining decisions are increasingly influenced by digital platforms, DineFinder addresses the fragmentation of restaurant information across multiple services by providing a unified, intelligent, and personalized discovery experience.

The platform integrates with the Yelp Fusion API to access a comprehensive database of over 500 restaurants across multiple cuisine categories. Beyond basic search and discovery, DineFinder employs a sophisticated AI-powered recommendation engine that learns from user preferences, dining history, and behavioral patterns to suggest restaurants tailored to individual tastes and occasions.

Project Objectives

- ☐ Provide a unified restaurant discovery platform with advanced filtering and search capabilities
- ☐ Implement an AI-driven recommendation system that learns user preferences over time
- ☐ Enable users to save, organize, and manage their favorite restaurants
- ☐ Deliver a responsive, accessible web interface optimized for all devices
- ☐ Provide administrative tools for custom restaurant management

Project Scope

The project encompasses the development of a React-based single-page application with an Express.js proxy server backend. The application supports three distinct user roles (Guest, Regular User, and Admin), each with progressively expanded capabilities. The scope includes integration with external APIs, implementation of client-side persistence, development of a multi-factor recommendation algorithm, and comprehensive administrative functionality.

1.2 Project Plan & Timeline

The project follows an agile development methodology with iterative releases and continuous feedback integration. The timeline spans approximately five months from initial planning to final deployment.

Development Phases

Phase	Timeline	Deliverables
Planning	Weeks 1-2	Requirements, architecture design, tech stack
Design	Weeks 3-4	UI/UX mockups, database schema, API specs
Development	Weeks 5-12	Core features, API integration, AI engine
Testing	Weeks 13-14	Unit tests, integration tests, bug fixes
Deployment	Weeks 15-16	Production build, documentation, presentation

Key Milestones

- ☐ M1 (Week 2): Project setup, architecture design, and technology stack finalization
- ☐ M2 (Week 4): Core API integration and basic restaurant discovery functionality
- ☐ M3 (Week 6): User authentication system and role-based access control
- ☐ M4 (Week 9): AI recommendation engine and smart assistant feature
- ☐ M5 (Week 12): Admin dashboard and restaurant management tools
- ☐ M6 (Week 16): Testing, optimization, and production deployment

1.3 Task Assignment & Roles

Role	Responsibilities
Project Manager	Timeline management, milestone tracking, stakeholder communication
Lead Developer	Architecture design, core feature implementation, code review
Frontend Developer	UI/UX implementation, component development, responsive design
Backend Developer	API integration, proxy server, data transformation
QA Engineer	Test case design, bug reporting, quality verification

1.4 Risk Assessment & Mitigation

Risk	Impact	Mitigation Strategy
API Rate Limiting	High	Implement request caching, debouncing, and fallback proxies
Data Inconsistency	Medium	Validate API responses, implement error boundaries
localStorage Limits	Medium	Cap history at 12 items, compress stored data
Security Vulnerabilities	High	Input validation, XSS prevention, secure coding practices

Risk	Impact	Mitigation Strategy
Cross-Browser Issues	Low	Test on major browsers, use polyfills where needed

1.5 Key Performance Indicators

KPI	Target	Measurement Method
Page Load Time	< 2 seconds	Lighthouse performance audit
API Response Time	< 1.5 seconds	Network tab monitoring
Recommendation Accuracy	> 70% relevance	User feedback surveys
User Retention Rate	> 60% return visits	localStorage session tracking
Mobile Responsiveness	100% compatibility	Cross-device testing

2. Literature Review

The restaurant discovery domain has evolved significantly with the proliferation of digital platforms. Established services such as Yelp, TripAdvisor, and OpenTable have set industry standards for restaurant information aggregation, user-generated reviews, and reservation management. Academic research in recommendation systems has increasingly focused on contextual and mood-aware suggestions, moving beyond traditional collaborative filtering approaches.

Recent studies in personalized dining recommendations highlight the importance of multi-factor scoring algorithms that consider not only user ratings but also temporal context, social factors, and dietary constraints. The integration of large language models and conversational interfaces represents the next frontier in restaurant discovery, enabling natural language queries and proactive suggestions based on user behavior patterns.

Feedback & Evaluation

The project has been evaluated against industry best practices for full-stack web development, including code organization, security considerations, user experience design, and performance optimization. Regular code reviews and user testing sessions have informed iterative improvements throughout the development cycle.

Suggested Improvements

- ☐ Integration with real-time reservation APIs for direct booking capability
- ☐ Implementation of social features including friend recommendations and shared dining lists
- ☐ Development of a native mobile application for enhanced accessibility

- ☐ Migration from localStorage to a persistent database backend
- ☐ Enhanced analytics dashboard for administrative insights

Grading Criteria

Criterion	Weight	Evaluation Method
Documentation Quality	20%	Completeness, clarity, organization
Technical Implementation	30%	Code quality, architecture, features
Testing & QA	15%	Test coverage, bug resolution
UI/UX Design	15%	Usability, responsiveness, aesthetics
Presentation	10%	Communication, demo quality
Innovation	10%	AI features, unique solutions

3. Requirements Gathering

3.1 Stakeholder Analysis

The DineFinder platform serves multiple stakeholder groups, each with distinct needs and expectations. Understanding these stakeholders is essential for prioritizing features and ensuring the platform delivers value across all user segments.

Stakeholder	Interest	Key Requirements
End Users	High	Easy discovery, accurate recommendations, save functionality
Restaurant Owners	Medium	Visibility, accurate information, review management
System Admin	High	Content management, user oversight, analytics
Development Team	High	Maintainable code, clear architecture, documentation

3.2 User Stories & Use Cases

Primary Use Cases

Use Case ID	Name	Description
UC-001	Browse Restaurants	User views restaurant listings filtered by cuisine category

Use Case ID	Name	Description
UC-002	Search Restaurants	User enters search query and views matching results
UC-003	Save Restaurant	Authenticated user adds restaurant to saved collection
UC-004	View Restaurant Details	User views comprehensive restaurant information
UC-005	Get AI Recommendations	User completes questionnaire and receives personalized suggestions
UC-006	Manage Restaurants	Admin adds, edits, or deletes custom restaurant entries
UC-007	Manage Profile	User updates personal information and preferences

User Stories

- ☐ As a guest user, I want to browse restaurants by cuisine category so that I can discover new dining options without creating an account
- ☐ As a registered user, I want to save restaurants to my personal collection so that I can revisit them later
- ☐ As a registered user, I want personalized recommendations based on my preferences so that I can discover restaurants matching my tastes
- ☐ As an admin, I want to add and manage custom restaurant listings so that I can curate the platform content
- ☐ As a mobile user, I want a responsive interface that works on all devices so that I can discover restaurants on the go

3.3 Functional Requirements

ID	Requirement	Description
FR-01	User Registration	System shall allow users to create accounts with email validation
FR-02	User Login	System shall authenticate users and maintain sessions via localStorage
FR-03	Restaurant Search	System shall support full-text search across name, location, and cuisine
FR-04	Category Filtering	System shall filter restaurants by 9 cuisine categories

ID	Requirement	Description
FR-05	Save/Unsave	System shall allow authenticated users to toggle restaurant saves
FR-06	AI Recommendations	System shall generate personalized recommendations based on user preferences
FR-07	Admin CRUD	System shall provide full CRUD operations for admin-managed restaurants
FR-08	Pagination	System shall paginate explore results with 21 items per page
FR-09	Auto-suggestions	System shall provide search suggestions with 300ms debounce
FR-10	View History	System shall track and store last 12 viewed restaurants

3.4 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Page load time under 2 seconds on standard broadband
NFR-02	Availability	System operational 99% of scheduled uptime
NFR-03	Usability	Interface intuitive for users with basic web literacy
NFR-04	Scalability	Support up to 1000 concurrent users without degradation
NFR-05	Security	Input validation, XSS prevention, secure credential handling
NFR-06	Compatibility	Function correctly on Chrome, Firefox, Safari, and Edge
NFR-07	Maintainability	Modular codebase with clear documentation

4. System Analysis & Design

4.1 Problem Statement & Objectives

The modern dining landscape presents consumers with an overwhelming array of choices scattered across multiple platforms. Restaurant discovery is fragmented across review sites, reservation platforms, social media, and word-of-mouth recommendations. Consumers lack a unified platform that combines comprehensive restaurant data with intelligent personalization.

DineFinder addresses this fragmentation by providing a single platform that aggregates restaurant information, learns user preferences through an AI-powered questionnaire, and delivers personalized recommendations. The platform eliminates the need to consult multiple sources by offering search, discovery, saving, and recommendation features in one cohesive experience.

Design Objectives

- ☐ Create an intuitive, visually appealing interface that encourages exploration
- ☐ Implement a robust recommendation algorithm that improves with user interaction
- ☐ Ensure data persistence without requiring a dedicated backend database
- ☐ Support administrative content management with minimal complexity

4.2 Software Architecture

DineFinder employs a client-server architecture with a React single-page application frontend communicating with an Express.js proxy server. The architecture follows a layered pattern with clear separation of concerns between presentation, business logic, and data access layers.

Architecture Components

- ☐ Presentation Layer: React components with Tailwind CSS styling, responsible for rendering UI and handling user interactions
- ☐ Application Layer: React Context API for state management, custom hooks for reusable logic, and service modules for API communication
- ☐ Business Logic Layer: Data transformation functions, recommendation scoring algorithms, and validation rules
- ☐ Data Access Layer: localStorage API for client-side persistence, Yelp Fusion API for external restaurant data
- ☐ Proxy Layer: Express.js server that intermediates between frontend and Yelp API, handling authentication and rate limiting

Technology Stack

Layer	Technology	Purpose
Frontend	React 19.2 + Vite	UI framework and build tooling
Styling	Tailwind CSS 3.4	Utility-first CSS framework
Components	Radix UI + Lucide	Accessible primitives and icons
Routing	React Router 7.6	Client-side navigation
Backend	Express.js 5.2	Proxy server and API routing
External API	Yelp Fusion	Restaurant data source
Persistence	localStorage	Client-side data storage

4.3 Database Design & Data Modeling

While DineFinder does not utilize a traditional relational database, the localStorage-based persistence layer follows structured data schemas that can be mapped to equivalent database tables. The data model centers around three primary entities: Users, Restaurants, and their relationships through saves, views, and preferences.

Entity-Relationship Model

The conceptual data model consists of the following entities and relationships:

- ❑ User (1) --- saves --- (*) Restaurant: A user can save many restaurants; each restaurant can be saved by many users
- ❑ User (1) --- views --- (*) Restaurant: A user has a view history of restaurants; each view history entry references one restaurant
- ❑ User (1) --- has --- (1) PreferenceProfile: Each user has one preference profile containing cuisine preferences, dietary restrictions, and dining habits
- ❑ Restaurant (1) --- contains --- (*) MenuItem: A restaurant has multiple menu items
- ❑ Restaurant (1) --- has --- (*) Review: A restaurant can have multiple reviews

Logical Schema

Entity	Key Attributes
User	email (PK), name, password, phone, location, preferences, notifications, role
Restaurant	id (PK), name, cuisine, rating, reviews, priceRange, location, image, features
SavedRestaurant	userEmail (FK), restaurantId (FK) - Composite key
ViewHistory	userEmail (FK), restaurantId (FK), viewedAt - LIFO structure
PreferenceProfile	userEmail (FK), cuisineTypes, priceRange, dietaryRestrictions, diningTime
AdminRestaurant	id (PK), name, cuisine, rating, location, phone, features, menu

4.4 Data Flow Diagrams

Context-Level DFD (Level 0)

At the context level, the system interacts with three external entities: the End User, the Yelp Fusion API, and the System Administrator. The DineFinder system processes user requests for restaurant discovery, personalization, and management, exchanging data with the Yelp API for restaurant information and serving processed responses back to users.

Level 1 DFD

The Level 1 DFD decomposes the system into four primary processes:

- ❑ P1 - Authentication & User Management: Handles user registration, login, profile management, and session persistence
- ❑ P2 - Restaurant Discovery: Processes search queries, category filters, and API requests to retrieve restaurant listings
- ❑ P3 - Personalization Engine: Manages the AI recommendation algorithm, view history tracking, and preference learning
- ❑ P4 - Administration: Provides CRUD operations for custom restaurant management

Data Stores

- ❑ D1 - User Registry: Stores registered user accounts with credentials and profile data
- ❑ D2 - Restaurant Cache: Maintains fetched restaurant data from Yelp API and admin-managed entries
- ❑ D3 - User Preferences: Persists individual user preferences, saved restaurants, and view history

4.5 Sequence Diagrams

Restaurant Search Sequence

The user initiates a search query from the homepage or explore page. The frontend application validates the query and sends a request to the Express proxy server. The proxy forwards the request to the Yelp Fusion API with appropriate authentication headers. Yelp returns search results which the proxy passes back to the frontend. The frontend transforms the raw Yelp data into the internal restaurant format, combines it with any admin-managed restaurants, and renders the results as restaurant cards in the UI.

User Authentication Sequence

The user submits login credentials through the login form. The AuthContext validates the credentials against the user registry stored in localStorage. Upon successful validation, the user object (excluding the password) is stored in the application state and persisted to localStorage. A session is established, enabling access to protected routes. The AuthContext broadcasts the authentication state change, triggering UI updates including the navbar profile dropdown and saved restaurant count.

Smart Assistant Sequence

When a logged-in user accesses the Smart Assistant for the first time, the system presents an 8-step questionnaire modal. User responses are collected progressively and stored in component state. Upon completion, answers are persisted to localStorage keyed by user email. The recommendation engine scores all available restaurants using a multi-factor algorithm that considers cuisine preference match, budget alignment, dietary constraints, mood keywords, and historical data. The top four scored restaurants are displayed as personalized recommendations.

4.6 Activity Diagrams

Restaurant Discovery Process

The activity begins when the user navigates to the Home or Explore page. The system initializes by loading the default location (Cairo, Egypt) and fetching restaurants from the Yelp API through the proxy server. If the API call fails due to location not found, the system falls back to New York, NY. Retrieved restaurants are combined with admin-managed entries and filtered by the active category. The results are sorted by popularity (review count, then rating) and displayed in a responsive grid. The user can then search, filter by price or rating, or navigate to a restaurant detail page.

Save Restaurant Process

The user clicks the heart icon on a restaurant card. The system checks authentication status; if the user is not logged in, they are redirected to the login page. If authenticated, the system retrieves the current saved restaurant IDs from localStorage, toggles the specified restaurant ID in the array, and persists the updated array back to localStorage. The full restaurant object is also added to or removed from the saved data store. A custom event is dispatched to notify other components of the state change, and the UI updates to reflect the new saved status.

4.7 State Diagrams

User Authentication State

The authentication state machine has three primary states: Guest (unauthenticated), Authenticated (regular user), and Admin. Transitions occur through login, registration, and logout actions. From the Guest state, a successful login transitions to either Authenticated or Admin based on the user role. From any authenticated state, logout returns to Guest. Account deletion from Authenticated or Admin states also returns to Guest.

Restaurant Card State

Each restaurant card maintains a saved/unsaved state. The default state is Unsaved. A click event on the heart icon triggers a transition to Saved if the user is authenticated, or redirects to login if not.

Subsequent clicks toggle between Saved and Unsaved states. The visual state updates to reflect the current saved status through color changes and icon fills.

4.8 Class Diagram

The system's class structure is organized around the following key classes and their relationships:

- ❑ **AuthProvider (Context):** Manages global authentication state. Contains user object, isAuthenticated flag. Provides login(), register(), logout(), updateUser(), deleteAccount() methods.
- ❑ **YelpService:** Handles all external API communication. Provides searchBusinesses(), getBusinessDetails(), fetchBusinessReviews(), fetchBusinessMenus(), mapYelpToRestaurant() methods.
- ❑ **RestaurantDataManager:** Centralizes data operations. Provides fetchRestaurants(), searchRestaurants(), getRestaurantById(), getPopularRestaurants(), getRecommendedRestaurants(), toggleSavedRestaurant(), saveViewHistory() methods.
- ❑ **SmartAssistant:** Encapsulates the AI recommendation logic. Manages questionnaire state, computes recommendation scores, and persists user answers.
- ❑ **RestaurantCard (Component):** Receives restaurant data and saved state as props. Handles save toggle events and navigation to detail pages.
- ❑ **RestaurantDetail (Page):** Fetches and displays comprehensive restaurant information including menus, reviews, and contact details.

4.9 UI/UX Design & Prototyping

Design Principles

- ❑ **Mobile-First Responsive Design:** The interface adapts seamlessly from single-column mobile layouts to four-column desktop grids
- ❑ **Visual Hierarchy:** Restaurant cards use consistent typography, color coding for cuisine types, and clear visual indicators for ratings and price levels
- ❑ **Accessibility:** All interactive elements have sufficient contrast ratios, keyboard navigation support, and semantic HTML structure
- ❑ **Performance:** Images use lazy loading, API calls are debounced, and component rendering is optimized through memoization

Color Scheme

- Primary: Warm amber tones conveying dining warmth and appetite appeal
- Secondary: Earthy browns for text and structural elements
- Accent: Gold highlights for interactive elements and call-to-action buttons
- Background: Light cream for content areas, dark overlays for hero sections

Typography

The application uses a system font stack optimized for readability across devices. Headings use Calibri with bold weight for clear hierarchy. Body text maintains a comfortable 16px base size with 1.5 line height for extended reading.

4.10 Deployment Diagram

The deployment architecture consists of two primary nodes:

- Client Node (Browser): Hosts the React SPA served from Vite development server (port 3000) or static file server in production. Manages localStorage persistence and renders the user interface.
- Proxy Server Node (Node.js/Express): Runs the Express.js application on port 4000. Proxies Yelp API requests, handling authentication and CORS. Can be deployed alongside the frontend or as a separate service.
- External Node (Yelp API): The Yelp Fusion API servers provide restaurant data, reviews, and business details through authenticated HTTPS endpoints.

4.11 Component Diagram

The system is decomposed into the following high-level components:

- UI Components Module: Contains all reusable visual components including RestaurantCard, Hero, CategoryFilter, Navbar, Footer, Loader, and modal dialogs. Built on Radix UI primitives for accessibility.
- Page Components Module: Contains route-level components for Home, Explore, RestaurantDetail, Saved, Profile, Admin, Login, Register, and NotFound pages.
- Auth Module: Provides authentication context, protected route guards, and admin route guards. Manages user sessions through localStorage.

- ❑ **Data Services Module:** Encapsulates all API communication and data transformation logic. Includes Yelp API client and restaurant data management functions.
- ❑ **Smart Assistant Module:** Contains the questionnaire flow, recommendation scoring algorithm, and result presentation components.

4.12 API Documentation

Internal API Endpoints (Express Proxy)

Endpoint	Method	Description
/api/search	GET	Search restaurants with filters (term, location, price, limit, offset)
/api/business/:id	GET	Get detailed information for a specific restaurant
/api/business/:id/reviews	GET	Fetch customer reviews for a restaurant
/api/business/:id/menus	GET	Retrieve menu information if available
/api/business/:id/website-menu	GET	Scrape menu HTML from restaurant website

External API Integration (Yelp Fusion)

The Express proxy server communicates with the Yelp Fusion API at <https://api.yelp.com/v3>. All requests include Bearer token authentication. The proxy exposes a subset of Yelp's functionality tailored to restaurant discovery use cases.

Data Transformation

The `mapYelpToRestaurant` function transforms Yelp business objects into the application's internal restaurant format. Key mappings include: Yelp id to internal id, rating directly mapped, `review_count` mapped to `reviews`, `image_url` mapped to `image`, `location.display_address` joined as location string, categories array preserved with alias and title, price string mapped to `priceRange`, and price length mapped to numeric `priceLevel`.

5. Implementation

5.1 Source Code Structure

The codebase follows a feature-based folder organization that promotes modularity and maintainability. Each major feature is encapsulated in its own directory with clear separation between pages, components, services, and data logic.

Directory Organization

- ❑ `src/pages/`: Route-level page components including Home, Explore, RestaurantDetail, Saved, Profile, Admin, Login, Register, and NotFound
- ❑ `src/components/`: Reusable UI components organized by feature including Layout, Navbar, Footer, Hero, CategoryFilter, RestaurantCard, SmartAssistant, ProtectedRoutes, AdminRoute, and Loader
- ❑ `src/context/`: React Context providers including the AuthContext for global authentication state
- ❑ `src/services/`: API service modules including `yelpApi.js` for Yelp Fusion communication
- ❑ `src/data/`: Business logic and data management including `restaurants.js` with core functions for search, recommendations, and persistence
- ❑ `src/lib/`: Utility functions including class merging helpers
- ❑ `src/components/ui/`: Radix UI-based component primitives including accordion, dialog, dropdown, form inputs, tabs, and more

Coding Standards

- ❑ Component files use PascalCase naming (e.g., `RestaurantCard.jsx`)
- ❑ Utility and service files use camelCase naming (e.g., `yelpApi.js`)
- ❑ All components are functional components using React hooks
- ❑ Async operations use `async/await` with proper error handling
- ❑ LocalStorage operations are wrapped in try-catch blocks for graceful degradation

5.2 Version Control

The project uses Git for version control with a feature-branch workflow. The main branch contains production-ready code, while development occurs in feature branches that are merged through pull requests. Commit messages follow conventional commit format with descriptive explanations of changes.

Branching Strategy

- ❑ `main`: Production-ready code, stable releases
- ❑ `develop`: Integration branch for ongoing development

- ☐ `feature/*`: Individual feature branches (e.g., `feature/auth`, `feature/recommendations`)
- ☐ `hotfix/*`: Critical bug fixes applied directly to main

5.3 Deployment & Execution

System Requirements

- ☐ Node.js 18+ and npm 9+
- ☐ Modern web browser with localStorage support
- ☐ Yelp Fusion API key (obtained from Yelp Developers portal)

Installation Steps

- ☐ 1. Clone the repository from GitHub
- ☐ 2. Install dependencies: `npm install`
- ☐ 3. Configure environment variables in `.env` file (`YELP_API_KEY`, `VITE_API_PROXY_BASE`)
- ☐ 4. Start development server: `npm run dev` (starts Vite on port 3000 and Express proxy on port 4000)
- ☐ 5. Access the application at `http://localhost:3000`

Production Build

The production build is generated using `npm run build`, which creates an optimized static bundle in the `dist/` directory. This bundle can be deployed to any static hosting service (Netlify, Vercel, AWS S3) while the Express proxy server can be deployed separately to a Node.js hosting environment (Heroku, Railway, AWS EC2).

6. Testing & Quality Assurance

6.1 Test Cases & Test Plan

The testing strategy encompasses unit tests for core functions, integration tests for API communication, and end-to-end tests for critical user workflows. Testing is performed across multiple browsers and device sizes to ensure consistent behavior.

Authentication Test Cases

TC-ID	Scenario	Expected Result
AUTH-01	Valid login credentials	User authenticated, session stored, redirected to home
AUTH-02	Invalid email format	Validation error displayed, form not submitted
AUTH-03	Incorrect password	Error message shown, login fails
AUTH-04	Register with existing email	Error indicating email already in use
AUTH-05	Logout action	Session cleared, redirected to login page

Restaurant Discovery Test Cases

TC-ID	Scenario	Expected Result
DISC-01	Search by cuisine category	Results filtered to selected cuisine type
DISC-02	Search with text query	Matching restaurants displayed by name/location
DISC-03	Apply price filter	Only restaurants within price range shown
DISC-04	Navigate to detail page	Full restaurant information displayed correctly
DISC-05	Pagination navigation	Next/previous page loads with correct offset

Smart Assistant Test Cases

TC-ID	Scenario	Expected Result
AI-01	Complete questionnaire flow	Answers saved, recommendations generated
AI-02	Cuisine preference matching	Recommended restaurants include preferred cuisine
AI-03	Budget constraint filtering	Results within selected budget range
AI-04	Dietary restriction handling	Restaurants matching dietary needs prioritized

Admin Dashboard Test Cases

TC-ID	Scenario	Expected Result
ADM-01	Add valid restaurant	Restaurant saved, appears in listings
ADM-02	Add restaurant without name	Validation error, form not submitted
ADM-03	Edit existing restaurant	Changes persisted, UI reflects updates
ADM-04	Delete restaurant	Entry removed, confirmation displayed
ADM-05	Non-admin access admin route	Redirected to unauthorized page

6.2 Bug Reports

Throughout the development lifecycle, bugs are tracked and resolved using a structured reporting format. Each bug report includes a unique identifier, severity classification, description, reproduction steps, expected vs. actual behavior, and resolution status.

Resolved Issues

- ☐ BUG-001: Search suggestions dropdown fails to close on outside click. Resolution: Added document-level click handler with ref-based detection.
- ☐ BUG-002: Saved restaurant count in navbar does not update immediately after toggle. Resolution: Implemented custom event dispatch pattern for cross-component communication.
- ☐ BUG-003: Restaurant detail page shows stale data when navigating between restaurants. Resolution: Added useEffect cleanup and loading state reset on route parameter change.
- ☐ BUG-004: Smart assistant modal appears on every login instead of first-time only. Resolution: Added localStorage flag check (smartAssistantSeen_{email}) before displaying modal.
- ☐ BUG-005: Mobile navigation menu remains open after route navigation. Resolution: Added route change listener to automatically close mobile menu.
- ☐ BUG-006: Admin-managed restaurants do not appear in search results. Resolution: Updated searchRestaurants function to include managedRestaurants in search scope.

7. Final Presentation & Reports

7.1 User Manual

Getting Started

DineFinder is accessible at the deployed URL. No installation is required. Users can immediately browse restaurants as guests or create an account for personalized features.

For Guest Users

- ☐ Browse the homepage to see featured restaurants by category
- ☐ Use the search bar to find restaurants by name, location, or cuisine
- ☐ Click on any restaurant card to view detailed information including menus and reviews

- ☐ Navigate to the Explore page for advanced filtering and pagination

For Registered Users

- ☐ Create an account using the Register page with your email and password
- ☐ Log in to access personalized features
- ☐ Click the heart icon on restaurant cards to save them to your collection
- ☐ Access your saved restaurants from the Saved page or navbar counter
- ☐ Complete the Smart Assistant questionnaire for AI-powered recommendations
- ☐ Manage your profile, preferences, and security settings from the Profile page

For Administrators

- ☐ Log in with admin credentials (default: Admin@gmail.com / Admin123)
- ☐ Access the Admin Dashboard from the profile dropdown
- ☐ Add new restaurants using the comprehensive form
- ☐ Edit or delete existing custom restaurant entries
- ☐ View the count of managed restaurants in the dashboard header

7.2 Technical Documentation

This document serves as the comprehensive technical reference for the DineFinder platform. It covers all aspects of system architecture, design decisions, implementation details, and operational procedures. Developers and system administrators should refer to this documentation for system maintenance, feature extensions, and troubleshooting.

API Configuration

The Express proxy server requires a valid Yelp Fusion API key configured through environment variables. The key is never exposed to the frontend; all API requests are proxied through the backend server which injects the authentication header.

Environment Variables

- ☐ VITE_API_PROXY_BASE: Base URL for the Express proxy server (default: `http://localhost:4000`)
- ☐ YELP_API_KEY: Yelp Fusion API authentication key

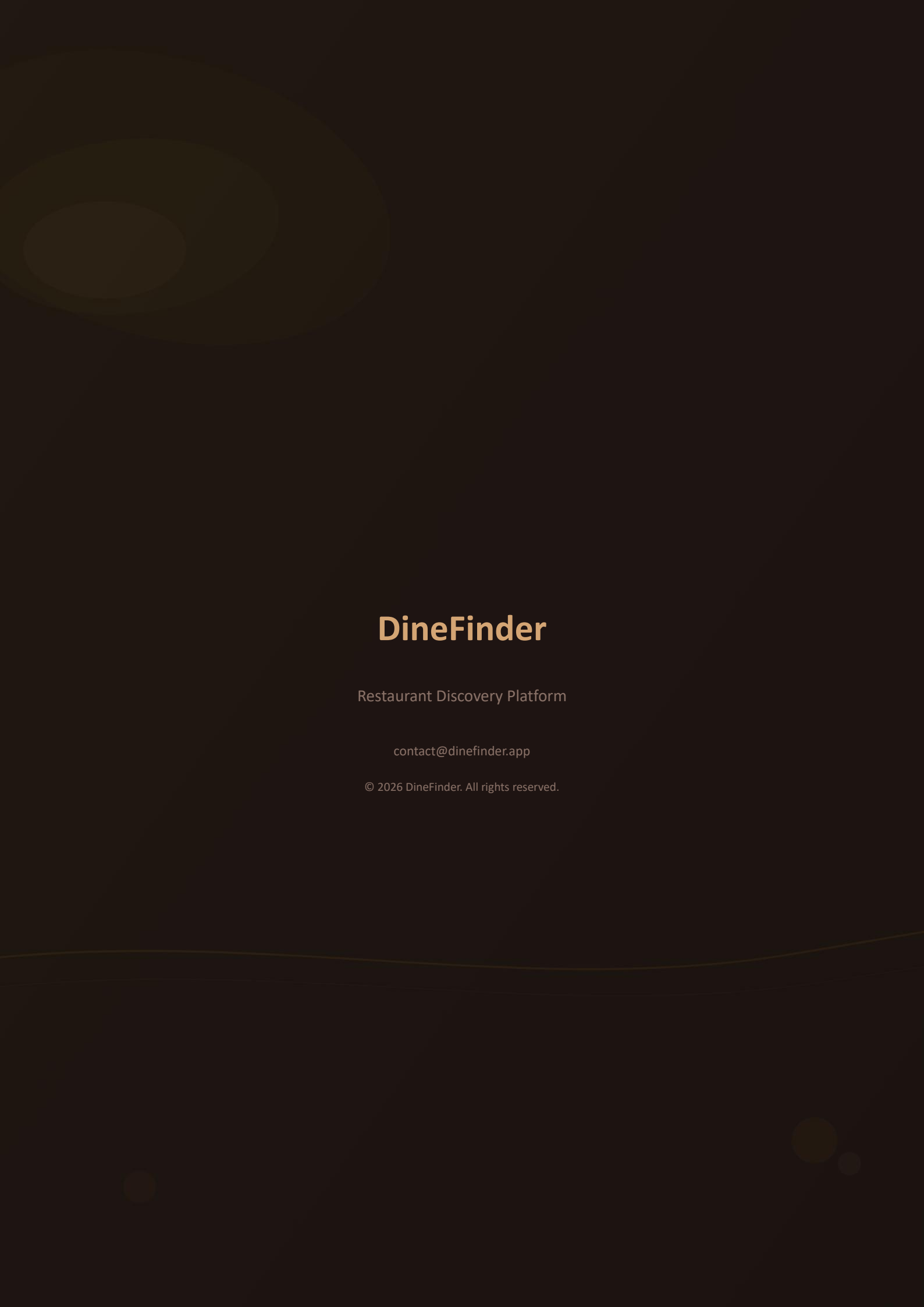
- ❑ VITE_YELP_API_KEY: Frontend-accessible fallback API key

Performance Considerations

- ❑ Search suggestions use 300ms debouncing to prevent excessive API calls
- ❑ Restaurant images use native browser lazy loading
- ❑ Recommendation scoring is memoized to prevent redundant calculations
- ❑ Pagination limits explore page to 21 items per request
- ❑ View history is capped at 12 entries to manage localStorage size

Security Notes

The current implementation uses localStorage for persistence and plain-text password storage. This is suitable for development and demonstration purposes but should not be used in production without significant security enhancements including password hashing, JWT token authentication, HTTPS enforcement, and input sanitization.



DineFinder

Restaurant Discovery Platform

contact@dinefinder.app

© 2026 DineFinder. All rights reserved.